JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class Byte

java.lang.Object
    java.lang.Number
        java.lang.Byte

All Implemented Interfaces:

Serializable, Comparable<Byte>

```
public final class Byte
extends Number
implements Comparable<Byte>
```

The `Byte` class wraps a value of primitive type `byte` in an object. An object of type `Byte` contains a single field whose type is `byte`.

In addition, this class provides several methods for converting a `byte` to a `String` and a `String` to a `byte`, as well as other constants and methods useful when dealing with a `byte`.

Since:

`JDK1.1`

See Also:

`Number`, `Serialized Form`

Field Summary

Fields

Modifier and Type	Field	Description
`static int`	**BYTES**	

		The number of bytes used to represent a `byte` value in two's complement binary form.
`static byte`	**`MAX_VALUE`**	A constant holding the maximum value a `byte` can have, 2^7-1.
`static byte`	**`MIN_VALUE`**	A constant holding the minimum value a `byte` can have, -2^7.
`static int`	**`SIZE`**	The number of bits used to represent a `byte` value in two's complement binary form.
`static Class<Byte>`	**`TYPE`**	The `Class` instance representing the primitive type `byte`.

Constructor Summary

Constructors

Constructor	Description
`Byte``(byte value)`	Constructs a newly allocated `Byte` object that represents the specified `byte` value.
`Byte``(`**`String`**` s)`	Constructs a newly allocated `Byte` object that represents the `byte` value indicated by the `String` parameter.

Method Summary

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
`byte`	**`byteValue`**`()`	Returns the value of this `Byte` as a `byte`.
`static int`	**`compare`**`(byte x, byte y)`	Compares two `byte` values numerically.
`int`	**`compareTo`**`(`**`Byte`**` anotherByte)`	Compares two `Byte` objects numerically.

static **Byte**	**decode**(**String** nm)	Decodes a String into a Byte.
double	**doubleValue**()	Returns the value of this Byte as a double after a widening primitive conversion.
boolean	**equals**(**Object** obj)	Compares this object to the specified object.
float	**floatValue**()	Returns the value of this Byte as a float after a widening primitive conversion.
int	**hashCode**()	Returns a hash code for this Byte; equal to the result of invoking intValue().
static int	**hashCode**(byte value)	Returns a hash code for a byte value; compatible with Byte.hashCode().
int	**intValue**()	Returns the value of this Byte as an int after a widening primitive conversion.
long	**longValue**()	Returns the value of this Byte as a long after a widening primitive conversion.
static byte	**parseByte**(**String** s)	Parses the string argument as a signed decimal byte.
static byte	**parseByte**(**String** s, int radix)	Parses the string argument as a signed byte in the radix specified by the second argument.
short	**shortValue**()	Returns the value of this Byte as a short

		after a widening primitive conversion.
String	**toString**()	Returns a String object representing this Byte's value.
static **String**	**toString**(byte b)	Returns a new String object representing the specified byte.
static int	**toUnsignedInt**(byte x)	Converts the argument to an int by an unsigned conversion.
static long	**toUnsignedLong**(byte x)	Converts the argument to a long by an unsigned conversion.
static **Byte**	**valueOf**(byte b)	Returns a Byte instance representing the specified byte value.
static **Byte**	**valueOf**(**String** s)	Returns a Byte object holding the value given by the specified String.
static **Byte**	**valueOf**(**String** s, int radix)	Returns a Byte object holding the value extracted from the specified String when parsed with the radix given by the second argument.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Detail

MIN_VALUE

```
public static final byte MIN_VALUE
```

A constant holding the minimum value a `byte` can have, -2^7.

See Also:

Constant Field Values

MAX_VALUE

```
public static final byte MAX_VALUE
```

A constant holding the maximum value a `byte` can have, 2^7-1.

See Also:

Constant Field Values

TYPE

```
public static final Class<Byte> TYPE
```

The `Class` instance representing the primitive type `byte`.

SIZE

```
public static final int SIZE
```

The number of bits used to represent a `byte` value in two's complement binary form.

Since:

1.5

See Also:

Constant Field Values

BYTES

```
public static final int BYTES
```

The number of bytes used to represent a `byte` value in two's complement binary form.

Since:

1.8

See Also:

Constant Field Values

Constructor Detail

Byte

```
public Byte(byte value)
```

Constructs a newly allocated `Byte` object that represents the specified `byte` value.

Parameters:

`value - the value to be represented by the Byte.`

Byte

```
public Byte(String s)
     throws NumberFormatException
```

Constructs a newly allocated `Byte` object that represents the `byte` value indicated by the `String` parameter. The string is converted to a `byte` value in exactly the manner used by the `parseByte` method for radix 10.

Parameters:

`s - the String to be converted to a Byte`

Throws:

`NumberFormatException - If the String does not contain a parsable byte.`

See Also:

parseByte(java.lang.String, int)

Method Detail

toString

```
public static String toString(byte b)
```

Returns a new `String` object representing the specified `byte`. The radix is assumed to be 10.

Parameters:

`b - the byte to be converted`

Returns:

`the string representation of the specified byte`

See Also:

`Integer.toString(int)`

valueOf

```
public static Byte valueOf(byte b)
```

Returns a `Byte` instance representing the specified `byte` value. If a new `Byte` instance is not required, this method should generally be used in preference to the constructor `Byte(byte)`, as this method is likely to yield significantly better space and time performance since all byte values are cached.

Parameters:

`b - a byte value.`

Returns:

`a Byte instance representing b.`

Since:

`1.5`

parseByte

```
public static byte parseByte(String s,
                             int radix)
                      throws NumberFormatException
```

Parses the string argument as a signed `byte` in the radix specified by the second argument. The characters in the string must all be digits, of the specified radix (as determined by whether `Character.digit(char, int)` returns a nonnegative value) except that the first character may be an ASCII minus sign `'-'` (`'\u002D'`) to indicate a negative value or an ASCII plus sign `'+'` (`'\u002B'`) to indicate a positive value. The resulting `byte` value is returned.

An exception of type `NumberFormatException` is thrown if any of the following situations occurs:

- The first argument is `null` or is a string of length zero.
- The radix is either smaller than `Character.MIN_RADIX` or larger than `Character.MAX_RADIX`.

- Any character of the string is not a digit of the specified radix, except that the first character may be a minus sign `'-'` (`'\u002D'`) or plus sign `'+'` (`'\u002B'`) provided that the string is longer than length 1.
- The value represented by the string is not a value of type `byte`.

Parameters:

`s - the String containing the byte representation to be parsed`

`radix - the radix to be used while parsing s`

Returns:

`the byte value represented by the string argument in the specified radix`

Throws:

`NumberFormatException - If the string does not contain a parsable byte.`

parseByte

```
public static byte parseByte(String s)
                      throws NumberFormatException
```

Parses the string argument as a signed decimal `byte`. The characters in the string must all be decimal digits, except that the first character may be an ASCII minus sign `'-'` (`'\u002D'`) to indicate a negative value or an ASCII plus sign `'+'` (`'\u002B'`) to indicate a positive value. The resulting `byte` value is returned, exactly as if the argument and the radix 10 were given as arguments to the `parseByte(java.lang.String, int)` method.

Parameters:

`s - a String containing the byte representation to be parsed`

Returns:

`the byte value represented by the argument in decimal`

Throws:

`NumberFormatException - if the string does not contain a parsable byte.`

valueOf

```
public static Byte valueOf(String s,
                           int radix)
                    throws NumberFormatException
```

Returns a `Byte` object holding the value extracted from the specified `String` when parsed with the radix given by the second argument. The first

argument is interpreted as representing a signed `byte` in the radix specified by the second argument, exactly as if the argument were given to the `parseByte(java.lang.String, int)` method. The result is a `Byte` object that represents the `byte` value specified by the string.

In other words, this method returns a `Byte` object equal to the value of:

```
new Byte(Byte.parseByte(s, radix))
```

Parameters:

`s - the string to be parsed`

`radix - the radix to be used in interpreting s`

Returns:

`a Byte object holding the value represented by the string argument in the specified radix.`

Throws:

`NumberFormatException - If the String does not contain a parsable byte.`

valueOf

```
public static Byte valueOf(String s)
                    throws NumberFormatException
```

Returns a `Byte` object holding the value given by the specified `String`. The argument is interpreted as representing a signed decimal `byte`, exactly as if the argument were given to the `parseByte(java.lang.String)` method. The result is a `Byte` object that represents the `byte` value specified by the string.

In other words, this method returns a `Byte` object equal to the value of:

```
new Byte(Byte.parseByte(s))
```

Parameters:

`s - the string to be parsed`

Returns:

`a Byte object holding the value represented by the string argument`

Throws:

`NumberFormatException - If the String does not contain a parsable byte.`

decode

```
public static Byte decode(String nm)
                   throws NumberFormatException
```

Decodes a `String` into a `Byte`. Accepts decimal, hexadecimal, and octal numbers given by the following grammar:

> ***DecodableString:***
>
> $Sign_{opt}$ *DecimalNumeral*
>
> $Sign_{opt}$ `0x` *HexDigits*
>
> $Sign_{opt}$ `0X` *HexDigits*
>
> $Sign_{opt}$ `#` *HexDigits*
>
> $Sign_{opt}$ `0` *OctalDigits*
>
> ***Sign:***
>
> `-`
>
> `+`

DecimalNumeral, *HexDigits*, and *OctalDigits* are as defined in section 3.10.1 of *The Java™ Language Specification*, except that underscores are not accepted between digits.

The sequence of characters following an optional sign and/or radix specifier ("`0x`", "`0X`", "`#`", or leading zero) is parsed as by the `Byte.parseByte` method with the indicated radix (10, 16, or 8). This sequence of characters must represent a positive value or a `NumberFormatException` will be thrown. The result is negated if first character of the specified `String` is the minus sign. No whitespace characters are permitted in the `String`.

Parameters:

`nm - the String to decode.`

Returns:

`a Byte object holding the byte value represented by nm`

Throws:

`NumberFormatException - if the String does not contain a parsable byte.`

See Also:

`parseByte(java.lang.String, int)`

byteValue

```
public byte byteValue()
```

Returns the value of this `Byte` as a `byte`.

Overrides:

`byteValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type byte.

shortValue

```
public short shortValue()
```

Returns the value of this `Byte` as a `short` after a widening primitive conversion.

Overrides:

`shortValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type short.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

intValue

```
public int intValue()
```

Returns the value of this `Byte` as an `int` after a widening primitive conversion.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type int.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

longValue

```
public long longValue()
```

Returns the value of this `Byte` as a `long` after a widening primitive conversion.

Specified by:

`longValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type long.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

floatValue

```
public float floatValue()
```

Returns the value of this `Byte` as a `float` after a widening primitive conversion.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type float.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

doubleValue

```
public double doubleValue()
```

Returns the value of this `Byte` as a `double` after a widening primitive conversion.

Specified by:

`doubleValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type double.

See *The Java™ Language Specification*:

5.1.2 Widening Primitive Conversions

toString

```
public String toString()
```

Returns a `String` object representing this `Byte`'s value. The value is converted to signed decimal representation and returned as a string, exactly as if the `byte` value were given as an argument to the `toString(byte)` method.

Overrides:

`toString` in class `Object`

Returns:

a string representation of the value of this object in base 10.

hashCode

```
public int hashCode()
```

Returns a hash code for this `Byte`; equal to the result of invoking `intValue()`.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this Byte

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

hashCode

```
public static int hashCode(byte value)
```

Returns a hash code for a `byte` value; compatible with `Byte.hashCode()`.

Parameters:

`value - the value to hash`

Returns:

`a hash code value for a byte value.`

Since:

`1.8`

equals

```
public boolean equals(Object obj)
```

Compares this object to the specified object. The result is `true` if and only if the argument is not `null` and is a `Byte` object that contains the same `byte` value as this object.

Overrides:

`equals` in class `Object`

Parameters:

`obj - the object to compare with`

Returns:

`true if the objects are the same; false otherwise.`

See Also:

`Object.hashCode()`, `HashMap`

compareTo

```
public int compareTo(Byte anotherByte)
```

Compares two `Byte` objects numerically.

Specified by:

`compareTo` in interface `Comparable<Byte>`

Parameters:

`anotherByte - the Byte to be compared.`

Returns:

`the value 0 if this Byte is equal to the argument Byte; a value less than 0 if this Byte is numerically less than the argument`

Byte; and a value greater than 0 if this Byte is numerically greater than the argument Byte (signed comparison).

Since:

1.2

compare

```
public static int compare(byte x,
                          byte y)
```

Compares two byte values numerically. The value returned is identical to what would be returned by:

```
    Byte.valueOf(x).compareTo(Byte.valueOf(y))
```

Parameters:

x - the first byte to compare

y - the second byte to compare

Returns:

the value 0 if x == y; a value less than 0 if x < y; and a value greater than 0 if x > y

Since:

1.7

toUnsignedInt

```
public static int toUnsignedInt(byte x)
```

Converts the argument to an int by an unsigned conversion. In an unsigned conversion to an int, the high-order 24 bits of the int are zero and the low-order 8 bits are equal to the bits of the byte argument. Consequently, zero and positive byte values are mapped to a numerically equal int value and negative byte values are mapped to an int value equal to the input plus 2^8.

Parameters:

x - the value to convert to an unsigned int

Returns:

the argument converted to int by an unsigned conversion

Since:

1.8

toUnsignedLong

```
public static long toUnsignedLong(byte x)
```

Converts the argument to a `long` by an unsigned conversion. In an unsigned conversion to a `long`, the high-order 56 bits of the `long` are zero and the low-order 8 bits are equal to the bits of the `byte` argument. Consequently, zero and positive `byte` values are mapped to a numerically equal `long` value and negative `byte` values are mapped to a `long` value equal to the input plus 2^8.

Parameters:

`x - the value to convert to an unsigned long`

Returns:

`the argument converted to long by an unsigned conversion`

Since:

`1.8`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.